

Capítulo 4: The Mathematics of Biological Systems

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

11 de maio de 2026

Sumário I

- 1 Introdução
- 2 Sistemas de Equações Diferenciais
- 3 Análise de Estabilidade
- 4 Bifurcações
- 5 Oscilações e Ciclos Limite
- 6 Análise de Sensibilidade
- 7 Exercícios
- 8 Referências

Objetivos de Aprendizagem

- Dominar ferramentas matemáticas avançadas para sistemas biológicos
- Compreender sistemas de equações diferenciais ordinárias (EDOs)
- Realizar análise de estabilidade e linearização
- Estudar bifurcações e transições de comportamento
- Analisar oscilações e ciclos limite
- Aplicar análise de sensibilidade paramétrica

Por que Matemática Avançada?

A matemática é essencial para:

- Predizer comportamento dinâmico de sistemas
- Identificar transições críticas e pontos de decisão
- Compreender robustez e sensibilidade
- Projetar intervenções terapêuticas eficazes

Definição: Modelagem Matemática

A modelagem matemática traduz processos biológicos em equações que capturam a essência dinâmica do sistema, permitindo simulação, análise e predição quantitativas.

Níveis de Complexidade:

- Modelos lineares (simples)
- Sistemas não-lineares (realistas)
- Modelos estocásticos (com ruído)
- Modelos espaciais (PDEs)

Ferramentas Principais:

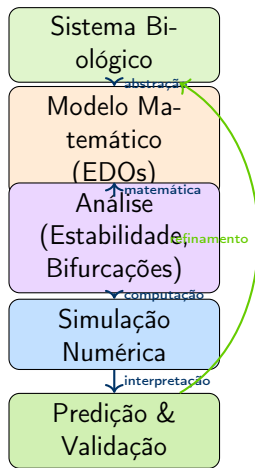
- Equações diferenciais
- Análise de estabilidade
- Teoria de bifurcações
- Métodos numéricos

Matemática como Linguagem da Biologia II

Importante!

Este capítulo foca em sistemas dinâmicos determinísticos descritos por EDOs

Visão Integrada: Do Modelo à Predição I



Tópicos Avançados deste Capítulo I

Parte I: Dinâmica

1. Sistemas de EDOs
2. Retratos de fase
3. Análise de estabilidade
4. Linearização e Jacobiano

Parte II: Transições

5. Bifurcações
6. Oscilações e ciclos limite

Parte III: Análise

7. Análise de sensibilidade
8. Exploração de espaço paramétrico

Aplicações

- Lotka-Volterra
- Van der Pol
- Modelos de regulação gênica
- Osciladores biológicos

Revisão: Equações Diferenciais Ordinárias I

Definição: Equação Diferencial Ordinária (EDO)

Uma EDO relaciona uma função desconhecida $y(t)$ com suas derivadas:

$$\frac{dy}{dt} = f(y, t)$$

onde t é a variável independente (geralmente tempo) e y é a variável dependente.

EDO de 1ª Ordem

$$\frac{dx}{dt} = -kx$$

Solução: $x(t) = x_0 e^{-kt}$
(decaimento exponencial)

EDO de 2ª Ordem

$$\frac{d^2x}{dt^2} + \omega^2 x = 0$$

Solução: $x(t) = A \cos(\omega t) + B \sin(\omega t)$
(oscilador harmônico)

Revisão: Equações Diferenciais Ordinárias II

Importante!

Sistemas biológicos geralmente requerem sistemas de EDOs acopladas!

Forma Geral

Um sistema de n equações diferenciais de primeira ordem:

$$\begin{aligned}\frac{dx_1}{dt} &= f_1(x_1, x_2, \dots, x_n, t) \\ \frac{dx_2}{dt} &= f_2(x_1, x_2, \dots, x_n, t) \\ &\vdots \\ \frac{dx_n}{dt} &= f_n(x_1, x_2, \dots, x_n, t)\end{aligned}$$

Forma Vetorial:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$$

Sistemas de EDOs Não-Lineares II

onde $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ e $\mathbf{f} = (f_1, f_2, \dots, f_n)^T$

Sistema Autônomo: quando $\mathbf{f}$ não depende explicitamente de t

Campos Vetoriais e Retratos de Fase I

Definição: Campo Vetorial

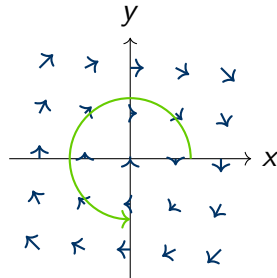
O lado direito do sistema $f(x)$ define um campo vetorial que associa a cada ponto do espaço de estados um vetor velocidade.

Exemplo Simples:

$$\begin{aligned}\frac{dx}{dt} &= y \\ \frac{dy}{dt} &= -x\end{aligned}$$

Interpretação:

- Em cada ponto (x, y) existe um vetor $(\dot{x}, \dot{y})$

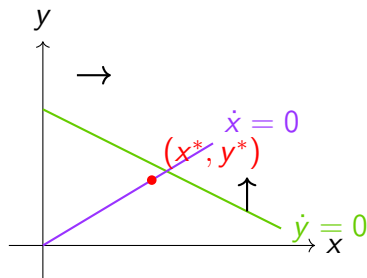


Definição: Nulclina

Curva no espaço de fase onde uma das derivadas é zero:

- **x-nulclina:** onde $\dot{x} = 0$ (movimento apenas vertical)
- **y-nulclina:** onde $\dot{y} = 0$ (movimento apenas horizontal)

Nulclinas II



Nulclinas se cruzam no equilíbrio

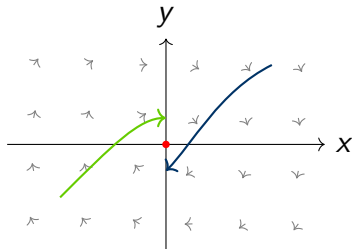
Importante!

Pontos de equilíbrio ocorrem na interseção das nulclinas!

Construindo Retratos de Fase

1. Calcular e plotar nulclinas ($\dot{x} = 0$ e $\dot{y} = 0$)
2. Identificar pontos de equilíbrio (interseções)
3. Desenhar campo de direções (vetores $(\dot{x}, \dot{y})$)
4. Integrar trajetórias numéricas a partir de condições iniciais
5. Analisar comportamento assintótico

Campos de Direção II



Resolvendo EDOs com SciPy I

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # Definir sistema de EDOs
6 def sistema(X, t, alpha, beta):
7     """Sistema de duas variaveis acopladas"""
8     x, y = X
9     dxdt = alpha*x - beta*x*y
10    dydt = -y + x*y
11    return [dxdt, dydt]
12
13 # Parametros
14 alpha = 1.0
15 beta = 0.5
```

Resolvendo EDOs com SciPy II

```
16
17 # Condicao inicial
18 X0 = [2.0, 1.0]
19
20 # Tempo
21 t = np.linspace(0, 20, 1000)
22
23 # Resolver
24 sol = odeint(sistema, X0, t, args=(alpha, beta))
25
26 # Plotar series temporal
27 plt.figure(figsize=(10, 4))
28 plt.plot(t, sol[:, 0], 'b-', label='x(t)')
29 plt.plot(t, sol[:, 1], 'r-', label='y(t)')
30 plt.xlabel('Tempo')
31 plt.ylabel('Concentracao')
```

Resolvendo EDOs com SciPy III

```
32 plt.legend()  
33 plt.grid(True)
```

Listing 1: Sistema de EDOs com `scipy.integrate`

Exemplo: Modelo Lotka-Volterra I

Sistema Predador-Presa

$$\begin{aligned}\frac{dx}{dt} &= \alpha x - \beta xy && \text{(presas)} \\ \frac{dy}{dt} &= \delta xy - \gamma y && \text{(predadores)}\end{aligned}$$

Parâmetros:

- α : taxa de crescimento das presas
- β : taxa de predação
- δ : eficiência de conversão
- γ : taxa de morte dos predadores

Exemplo: Modelo Lotka-Volterra II

Ponto de Equilíbrio:

$$x^* = \frac{\gamma}{\delta}, \quad y^* = \frac{\alpha}{\beta}$$

Importante!

Este sistema exibe oscilações periódicas (órbitas fechadas)!

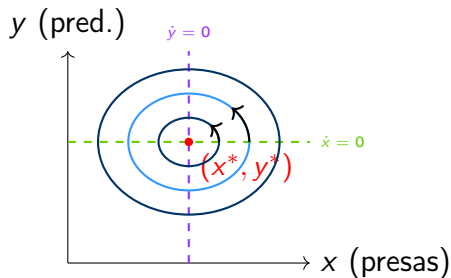
Retrato de Fase do Lotka-Volterra I

Características:

- Órbitas fechadas (periódicas)
- Centro estável em (x^*, y^*)
- Amplitude depende da condição inicial
- Não há atrator: órbitas neutras
- Sistema conservativo

Nulclinas:

- $\dot{x} = 0$: $y = \alpha/\beta$
- $\dot{y} = 0$: $x = \gamma/\delta$



Interpretação:

Populações oscilam ciclicamente com defasagem temporal

Simulando Lotka-Volterra I

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def lotka_volterra(X, t, alpha, beta, delta, gamma):
6     """Modelo predador-presa"""
7     x, y = X
8     dxdt = alpha*x - beta*x*y
9     dydt = delta*x*y - gamma*y
10    return [dxdt, dydt]
11
12 # Parametros
13 alpha, beta, delta, gamma = 1.0, 0.5, 0.5, 1.0
14
15 # Condições iniciais
```

Simulando Lotka-Volterra II

```
16 X0 = [2.0, 1.0]
17
18 # Tempo
19 t = np.linspace(0, 50, 1000)
20
21 # Resolver
22 sol = odeint(lotka_volterra, X0, t, args=(alpha, beta, delta,
      gamma))
23
24 # Plotar retrato de fase
25 plt.figure(figsize=(8, 6))
26 plt.plot(sol[:, 0], sol[:, 1], 'b-', linewidth=2)
27 plt.plot(gamma/delta, alpha/beta, 'ro', markersize=10, label='
      Equilibrio')
28 plt.xlabel('Presas (x)')
29 plt.ylabel('Predadores (y)')
```

Simulando Lotka-Volterra III

```
30 plt.legend()  
31 plt.grid(True)
```

Definição: Ponto de Equilíbrio (Ponto Fixo)

Um ponto $\mathbf{x}^*$ é um ponto de equilíbrio se:

$$\mathbf{f}(\mathbf{x}^*) = 0$$

Ou seja, todas as derivadas são zero: $\dot{x}_i = 0$ para todo i .

Pontos de Equilíbrio II

Interpretação Biológica

Pontos de equilíbrio representam:

- Estados estacionários do sistema
- Concentrações constantes ao longo do tempo
- Homeostase em sistemas biológicos
- Possíveis estados finais de longo prazo

Questão Central: O equilíbrio é estável ou instável?

- **Estável:** sistema retorna após pequenas perturbações
- **Instável:** sistema diverge após perturbações

Linearização em torno do Equilíbrio

Considere pequena perturbação $\xi(t) = x(t) - x^*$:

$$\frac{d\xi}{dt} = J(x^*)\xi$$

onde J é a matriz Jacobiana avaliada no equilíbrio.

Definição: Matriz Jacobiana

Matriz das derivadas parciais:

$$J_{ij} = \left. \frac{\partial f_i}{\partial x_j} \right|_{\mathbf{x}^*}$$

Para sistema 2D:

$$\mathbf{J} = \left[\begin{array}{cc} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{array} \right]_{\mathbf{x}^*}$$

Importante!

A estabilidade do equilíbrio $\mathbf{x}^*$ é determinada pelos autovalores λ_i da matriz Jacobiana $J(\mathbf{x}^*)$.

Critério de Estabilidade

Para cada autovalor $\lambda = a + bi$:

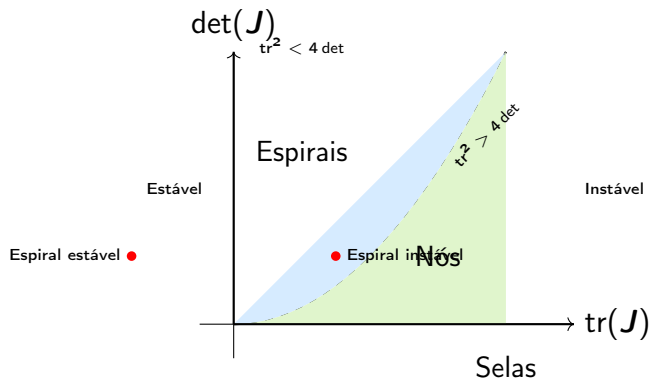
- Se $\text{Re}(\lambda_i) < 0$ para todos $i \Rightarrow$ **estável**
- Se $\text{Re}(\lambda_i) > 0$ para algum $i \Rightarrow$ **instável**
- Se $\text{Re}(\lambda_i) = 0$ para algum $i \Rightarrow$ **marginal** (análise não-linear necessária)

Equação Característica (2D):

$$\det(\mathbf{J} - \lambda \mathbf{I}) = 0 \quad \Rightarrow \quad \lambda^2 - \text{tr}(\mathbf{J})\lambda + \det(\mathbf{J}) = 0$$

onde $\text{tr}(\mathbf{J}) = J_{11} + J_{22}$ e $\det(\mathbf{J}) = J_{11}J_{22} - J_{12}J_{21}$

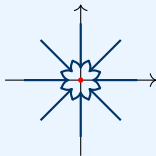
Classificação de Pontos Fixos em 2D I



Tipos de Pontos Fixos I

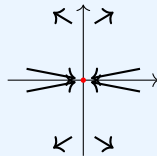
1. Nó Estável

- Ambos $\lambda_i < 0$ (reais)
- Convergência exponencial
- Sem oscilações



3. Ponto de Sela

- $\lambda_1 < 0 < \lambda_2$
- Instável (pelo menos uma direção diverge)



2. Nó Instável

- Ambos $\lambda_i > 0$ (reais)
- Divergência exponencial

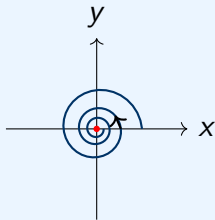
4. Centro

- $\lambda = \pm i\omega$ (imaginários puros)
- Órbitas fechadas

Espiraís: Estáveis vs. Instáveis I

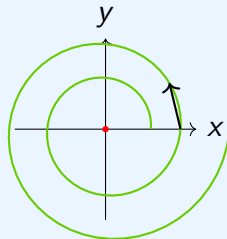
Espiral Estável

- $\lambda = a \pm bi$ com $a < 0$
- Convergência oscilatória
- Período: $T = 2\pi/|b|$
- Amortecimento: e^{at}



Espiral Instável

- $\lambda = a \pm bi$ com $a > 0$
- Divergência oscilatória
- Período: $T = 2\pi/|b|$
- Crescimento: e^{at}



Espiraís: Estáveis vs. Instáveis II

Importante!

Espiraís são comuns em sistemas biológicos com feedback negativo e atraso!

Teorema de Estabilidade de Lyapunov I

Definição: Estabilidade de Lyapunov

Um ponto de equilíbrio $\mathbf{x}^*$ é:

- **Estável:** se para todo $\epsilon > 0$ existe $\delta > 0$ tal que $\|\mathbf{x}(0) - \mathbf{x}^*\| < \delta$ implica $\|\mathbf{x}(t) - \mathbf{x}^*\| < \epsilon$ para todo $t \geq 0$
- **Assintoticamente estável:** se é estável e $\lim_{t \rightarrow \infty} \mathbf{x}(t) = \mathbf{x}^*$

Teorema de Estabilidade de Lyapunov II

Função de Lyapunov

Função escalar $V(\mathbf{x})$ tal que:

1. $V(\mathbf{x}^*) = 0$ e $V(\mathbf{x}) > 0$ para $\mathbf{x} \neq \mathbf{x}^*$
2. $\dot{V}(\mathbf{x}) \leq 0$ ao longo de trajetórias

Se existe tal V , então $\mathbf{x}^*$ é estável.

Se $\dot{V}(\mathbf{x}) < 0$ (estritamente), então é assintoticamente estável.

Interpretação: $V(\mathbf{x})$ é uma “energia” que sempre diminui

Calculando Jacobiano e Autovalores com Python I

```
1 import numpy as np
2 from scipy.linalg import eig
3 import sympy as sp
4
5 # Definir variaveis simbolicas
6 x, y = sp.symbols('x y')
7
8 # Definir sistema
9 f1 = x*(1 - x - 0.5*y)
10 f2 = y*(-0.75 + 0.5*x)
11
12 # Calcular Jacobiano simbolicamente
13 J = sp.Matrix([[sp.diff(f1, x), sp.diff(f1, y)],
14                [sp.diff(f2, x), sp.diff(f2, y)]])
15
```

Calculando Jacobiano e Autovalores com Python II

```
16 print("Jacobiano simbolico:")
17 print(J)
18
19 # Encontrar pontos de equilibrio
20 equilibria = sp.solve([f1, f2], [x, y])
21 print("\nPontos de equilibrio:", equilibria)
22
23 # Avaliar no equilibrio (exemplo: x=1.5, y=1)
24 x_star, y_star = 1.5, 1.0
25 J_numeric = np.array(J.subs([(x, x_star), (y, y_star)])) .astype(
    float)
26
27 # Calcular autovalores
28 eigenvalues, eigenvectors = eig(J_numeric)
29 print(f"\nAutovalores em ({x_star}, {y_star}):", eigenvalues)
30 print("Estavel?", all(np.real(eigenvalues) < 0))
```

Calculando Jacobiano e Autovalores com Python III

Exemplo: Análise de Estabilidade I

Sistema:

$$\begin{aligned}\frac{dx}{dt} &= x(1 - x - 0.5y) \\ \frac{dy}{dt} &= y(-0.75 + 0.5x)\end{aligned}$$

Ponto de Equilíbrio: $(x^*, y^*) = (1.5, 1)$

Jacobiano:

$$J = \begin{bmatrix} 1 - 2x - 0.5y & -0.5x \\ 0.5y & -0.75 + 0.5x \end{bmatrix}$$

Em $(1.5, 1)$:

$$J(1.5, 1) = \begin{bmatrix} -2.5 & -0.75 \\ 0.5 & 0 \end{bmatrix}$$

Exemplo: Análise de Estabilidade II

Autovalores: $\lambda = -1.25 \pm 0.61i$ (parte real negativa)

Importante!

O equilíbrio é uma **espiral estável!**

O que são Bifurcações? I

Definição: Bifurcação

Uma bifurcação é uma mudança qualitativa na dinâmica de um sistema quando um parâmetro atravessa um valor crítico. Novos comportamentos emergem, equilíbrios aparecem/desaparecem ou mudam de estabilidade.

O que são Bifurcações? II

Importância Biológica

Bifurcações explicam:

- Transições entre estados celulares (diferenciação)
- Surgimento de oscilações (ritmos circadianos)
- Switches biestáveis (decisões celulares)
- Transições de fase em populações
- Limiar para patologias (onset de doenças)

Parâmetro de bifurcação: μ (valor crítico: μ_c)

Bifurcação Saddle-Node (Fold) I

Equação Normal

$$\frac{dx}{dt} = \mu + x^2$$

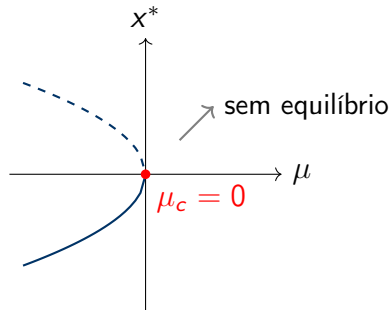
Análise:

- $\mu < 0$: dois equilíbrios
($x^* = \pm\sqrt{-\mu}$)
- $\mu = 0$: um equilíbrio
($x^* = 0$) - bifurcação!
- $\mu > 0$: nenhum equilíbrio

Interpretação:

- Colisão e aniquilação de equilíbrios

Diagrama de Bifurcação



Linha sólida: estável
Linha tracejada: instável

Bifurcação Saddle-Node (Fold) II

Importante!

Saddle-node é a forma mais comum de equilíbrios aparecerem/desaparecerem!

Bifurcação Transcrítica I

Equação Normal

$$\frac{dx}{dt} = \mu x - x^2$$

Equilíbrios:

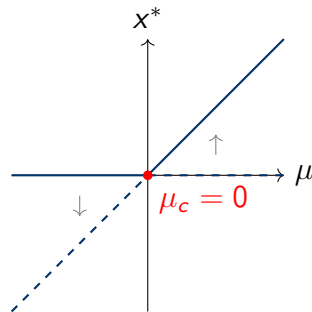
- $x^* = 0$ (sempre existe)
- $x^* = \mu$

Estabilidade:

- $\mu < 0$: $x = 0$ estável, $x = \mu$ instável
- $\mu > 0$: $x = 0$ instável, $x = \mu$ estável
- $\mu = 0$: troca de estabilidade!

Interpretação:

Diagrama de Bifurcação



Exemplo Biológico:

Transição de crescimento populacional

Bifurcação Pitchfork I

Supercrítica (Suave)

$$\frac{dx}{dt} = \mu x - x^3$$

Equilíbrios:

- $x^* = 0$
- $x^* = \pm\sqrt{\mu}$ (se $\mu > 0$)

Estabilidade:

- $\mu < 0$: só $x = 0$ (estável)
- $\mu > 0$: $x = 0$ instável, $x = \pm\sqrt{\mu}$ estáveis

x^*



Subcrítica (Abrupta)

$$\frac{dx}{dt} = \mu x + x^3$$

Equilíbrios:

- $x^* = 0$
- $x^* = \pm\sqrt{-\mu}$ (se $\mu < 0$)

Estabilidade:

- $\mu < 0$: $x = 0$ estável, $x = \pm\sqrt{-\mu}$ instáveis
- $\mu > 0$: $x = 0$ instável (divergência)

x^*

Bifurcação Pitchfork II

Importante!

Pitchfork supercrítica: transição suave. Subcrítica: salto catastrófico!

Bifurcação de Hopf I

Definição: Bifurcação de Hopf

Transição em que um equilíbrio estável perde estabilidade e dá origem a um ciclo limite periódico (oscilação).

Condições:

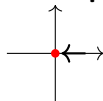
- Par de autovalores complexos conjugados: $\lambda = \alpha(\mu) \pm i\omega(\mu)$
- No ponto crítico μ_c : $\alpha(\mu_c) = 0$
- $\left. \frac{d\alpha}{d\mu} \right|_{\mu_c} > 0$

Tipos:

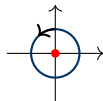
- **Supercrítica:** ciclo limite estável

Amplitude do ciclo:

Hopf Supercrítica



$\mu < \mu_c$
estável



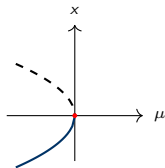
$\mu > \mu_c$
oscilação

Bifurcação de Hopf II

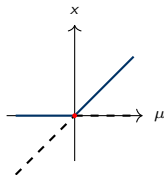
Importante!

Hopf é crucial para o surgimento de oscilações biológicas!

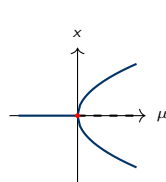
Diagramas de Bifurcação: Resumo Visual I



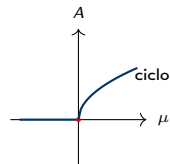
Saddle-node



Transcrítica



Pitchfork



Hopf

Gerando Diagramas de Bifurcação com Python I

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4
5 def pitchfork(x, mu):
6     """Bifurcacao pitchfork supercritica"""
7     return mu*x - x**3
8
9 # Varrer parametro mu
10 mu_vals = np.linspace(-2, 2, 100)
11 equilibria = []
12
13 for mu in mu_vals:
14     # Encontrar todos os equilíbrios
15     eq_list = []
```

Gerando Diagramas de Bifurcação com Python II

```
16     for x0 in [-2, 0, 2]: # tentativas iniciais
17         eq = fsolve(pitchfork, x0, args=(mu,))[0]
18         if abs(pitchfork(eq, mu)) < 1e-6: # verificar solucao
19             eq_list.append(eq)
20
21     # Remover duplicatas
22     eq_list = list(set(np.round(eq_list, 6)))
23
24     for eq in eq_list:
25         # Verificar estabilidade: derivada de f em x
26         stability = mu - 3*eq**2
27         if stability < 0: # estavel
28             equilibria.append((mu, eq, 'stable'))
29         else: # instavel
30             equilibria.append((mu, eq, 'unstable'))
31
```

Gerando Diagramas de Bifurcação com Python III

```
32 # Plotar diagrama
33 stable = [(m, x) for m, x, s in equilibria if s == 'stable']
34 unstable = [(m, x) for m, x, s in equilibria if s == 'unstable']
35
36 plt.plot([m for m, x in stable], [x for m, x in stable], 'b-',
          label='Estavel')
37 plt.plot([m for m, x in unstable], [x for m, x in unstable], 'r--',
          label='Instavel')
38 plt.axvline(0, color='k', linestyle=':', alpha=0.5)
39 plt.xlabel('$\mu$')
40 plt.ylabel('$x^*$')
41 plt.legend()
42 plt.title('Bifurcacao Pitchfork')
```

Soluções Periódicas I

Definição: Solução Periódica

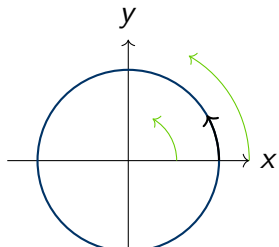
Uma solução $x(t)$ é periódica com período T se:

$$x(t + T) = x(t) \quad \forall t$$

No espaço de fase, corresponde a uma órbita fechada.

Tipos de Oscilações:

- **Linear:** oscilador harmônico (centro)
- **Não-linear:** ciclos limite
- **Forçada:** com input periódico



Ciclos Limite I

Definição: Ciclo Limite

Uma órbita periódica isolada para a qual trajetórias próximas convergem (estável) ou divergem (instável). Diferente de centro, é estruturalmente estável sob perturbações.

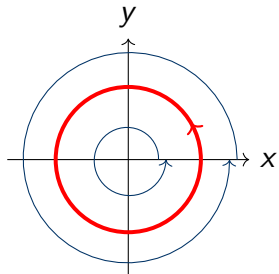
Propriedades:

- Órbita fechada isolada
- Amplitude e período fixos
- Atrator (se estável)
- Robusto a perturbações
- Não existe em sistemas lineares

Diferença de Centro:

- Centro: família de órbitas

Ciclo Limite Estável



Importante!

Ciclos limite são essenciais para modelar ritmos biológicos robustos!

Teorema de Poincaré-Bendixson I

Teorema de Poincaré-Bendixson

Para sistemas 2D ($\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$), se:

1. Existe uma região limitada R positivamente invariante
2. R não contém pontos de equilíbrio
3. Uma trajetória $\mathbf{x}(t)$ permanece em R para $t \rightarrow \infty$

Então $\mathbf{x}(t)$ converge para um ciclo limite periódico.

Importante!

Este teorema garante a existência de oscilações em sistemas 2D! Não vale para dimensões maiores (onde caos é possível).

Teorema de Poincaré-Bendixson II

Aplicação:

- Provar existência de oscilações
- Descartar caos em 2D
- Guiar busca numérica por ciclos

Oscilador de Van der Pol I

Equação de Van der Pol

$$\frac{d^2x}{dt^2} - \mu(1 - x^2)\frac{dx}{dt} + x = 0$$

onde $\mu > 0$ controla não-linearidade.

Sistema de primeira ordem:

$$\frac{dx}{dt} = y$$

$$\frac{dy}{dt} = \mu(1 - x^2)y - x$$

Características:

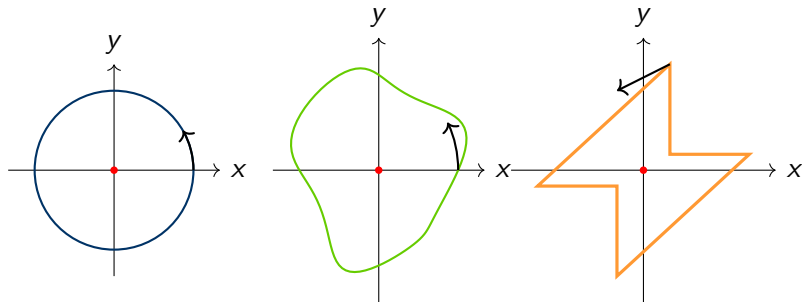
- μ pequeno: quase harmônico
- μ grande: oscilações de relaxação
- Único ciclo limite estável
- Equilíbrio instável na origem

Análise de Estabilidade:

$$J(0, 0) = \begin{bmatrix} 0 & 1 \\ -1 & \mu \end{bmatrix}$$

Autovalores: $\lambda = \frac{\mu \pm \sqrt{\mu^2 - 4}}{2}$
 $\text{Re}(\lambda) > 0$ se $\mu > 0 \Rightarrow$ instável

Retratos de Fase do Van der Pol I



$\mu = 0.5$ (quase senoidal)

$\mu = 2$ (não-linear)

$\mu = 5$ (relaxação)

Retratos de Fase do Van der Pol II

Importante!

Van der Pol é paradigmático para osciladores biológicos não-lineares!

Simulando Van der Pol I

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def van_der_pol(X, t, mu):
6     """Oscilador de Van der Pol"""
7     x, y = X
8     dxdt = y
9     dydt = mu*(1 - x**2)*y - x
10    return [dxdt, dydt]
11
12 # Parametros
13 mu = 2.0
14
15 # Condição inicial
```

Simulando Van der Pol II

```
16 X0 = [0.1, 0.1]
17
18 # Tempo
19 t = np.linspace(0, 50, 2000)
20
21 # Resolver
22 sol = odeint(van_der_pol, X0, t, args=(mu,))
23
24 # Plotar retrato de fase
25 plt.figure(figsize=(12, 5))
26
27 # Serie temporal
28 plt.subplot(1, 2, 1)
29 plt.plot(t, sol[:, 0], 'b-', label='x(t)')
30 plt.xlabel('Tempo')
31 plt.ylabel('x')
```

Simulando Van der Pol III

```
32 plt.grid(True)
33 plt.legend()
34
35 # Retrato de fase
36 plt.subplot(1, 2, 2)
37 plt.plot(sol[:, 0], sol[:, 1], 'r-', linewidth=0.5)
38 plt.plot(sol[0, 0], sol[0, 1], 'go', markersize=8, label='Inicial
    ')
39 plt.xlabel('x')
40 plt.ylabel('y')
41 plt.legend()
42 plt.grid(True)
```

Osciladores Biológicos I

Ritmos Circadianos

- Período $\sim 24h$
- Genes clock (Per, Cry, Bmal1)
- Feedback transcricional negativo
- Ciclo limite robusto

Oscilações Glicolíticas

- Período: ~ 5 min
- PFK (fosfofrutoquinase)
- Controle alostérico
- Coordenação metabólica

Oscilações de Cálcio

- Período: segundos a minutos
- IP3 e Ca^{2+} intracelular
- Sinalização celular
- Feedback positivo/negativo

Ciclo Celular

- Período: horas
- Ciclinas e CDKs
- Checkpoints
- Switches biestáveis + oscilador

Importante!

Todos esses sistemas exibem ciclos limite robustos a perturbações!

Análise de Sensibilidade Local I

Definição: Análise de Sensibilidade

Estudo de como mudanças nos parâmetros do modelo afetam as saídas (variáveis de estado, fluxos, propriedades emergentes).

Perguntas:

- Quais parâmetros são mais influentes?
- Quão confiáveis são as predições?
- Onde concentrar esforços experimentais?
- Quais parâmetros são alvos terapêuticos?

Tipos:

- **Local:** derivadas parciais em ponto nominal
- **Global:** exploração de todo espaço paramétrico
- **Temporal:** dependência com tempo
- **Estrutural:** topologia do modelo

Definição: Coeficiente de Sensibilidade Local

Derivada normalizada da saída y em relação ao parâmetro p :

$$S_{y,p} = \frac{\partial y}{\partial p} \cdot \frac{p}{y} = \frac{\partial \ln y}{\partial \ln p}$$

Mede a variação percentual em y quando p varia 1%.

Interpretação

- $|S_{y,p}| > 1$: alta sensibilidade (variação amplificada)
- $|S_{y,p}| < 1$: baixa sensibilidade (variação amortecida)
- $S_{y,p} > 0$: y aumenta com p
- $S_{y,p} < 0$: y diminui com p

Para EDOs: sensibilidade depende do tempo: $S_{y,p}(t)$

Equações de Sensibilidade I

Para sistema $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{p})$, definimos:

$$s_i(t) = \frac{\partial \mathbf{x}(t)}{\partial p_i}$$

Equação de Sensibilidade

Derivando o sistema em relação a p_i :

$$\frac{ds_i}{dt} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \mathbf{s}_i + \frac{\partial \mathbf{f}}{\partial p_i}$$

ou seja:

$$\frac{ds_i}{dt} = \mathbf{J}(\mathbf{x}(t)) \mathbf{s}_i + \mathbf{g}_i(\mathbf{x}(t))$$

onde $\mathbf{J}$ é o Jacobiano e $\mathbf{g}_i = \partial \mathbf{f} / \partial p_i$.

Condição inicial: $\mathbf{s}_i(0) = \partial \mathbf{x}_0 / \partial p_i$ (geralmente 0)

Equações de Sensibilidade III

Importante!

Resolver equações de sensibilidade junto com o sistema original!

Métodos Globais

Exploram todo o espaço paramétrico, não apenas um ponto:

1. Método de Morris (One-At-a-Time)

- Screening de parâmetros
- Elementary effects
- Identifica parâmetros importantes
- Computacionalmente eficiente
- Medidas: μ (efeito médio), σ (não-linearidade/interação)

2. Análise de Variância (Sobol)

- Decomposição da variância
- Índices de Sobol
- S_i : efeito principal de p_i
- S_{ij} : interações entre p_i e p_j
- S_T^i : efeito total (inclui interações)

Índices de Sobol

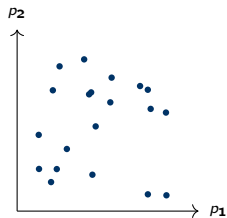
$$S_i = \frac{V[\mathbb{E}(Y|p_i)]}{V[Y]}, \quad S_T^i = \frac{\mathbb{E}[V(Y|p_{\sim i})]}{V[Y]}$$

onde $p_{\sim i}$ denota todos os parâmetros exceto p_i .

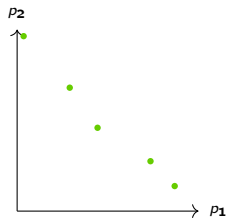
Técnicas de Amostragem

- **Monte Carlo:** amostragem aleatória
- **Latin Hypercube Sampling (LHS):** amostragem estratificada
- **Sobol sequences:** quasi-aleatória (baixa discrepância)

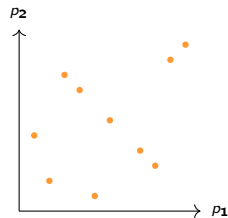
Exploração do Espaço Paramétrico II



Monte Carlo



Latin Hypercube



Sobol Sequence

Exemplo: Sensibilidade de Modelo Cinético I

Modelo: Reação enzimática simples

$$\frac{dx}{dt} = \frac{V_{max} \cdot S}{K_M + S} - k_{deg} \cdot x$$

Parâmetros: $V_{max} = 10$, $K_M = 5$, $k_{deg} = 0.5$, $S = 8$ (fixo)

Estado estacionário:

$$x^* = \frac{V_{max} \cdot S}{k_{deg}(K_M + S)} = \frac{10 \times 8}{0.5 \times 13} = 12.3$$

Exemplo: Sensibilidade de Modelo Cinético II

Sensibilidades em estado estacionário:

$$S_{x^*, V_{max}} = \frac{\partial x^*}{\partial V_{max}} \frac{V_{max}}{x^*} = 1$$

$$S_{x^*, k_{deg}} = \frac{\partial x^*}{\partial k_{deg}} \frac{k_{deg}}{x^*} = -1$$

$$S_{x^*, K_M} = \frac{\partial x^*}{\partial K_M} \frac{K_M}{x^*} = -\frac{K_M}{K_M + S} \approx -0.38$$

Importante!

x^* é mais sensível a V_{max} e k_{deg} do que a K_M !

Calculando Sensibilidade com Python I

```
1 import numpy as np
2 from scipy.integrate import odeint
3
4 def modelo_sistema(x, t, Vmax, Km, kdeg, S):
5     """Modelo com parametros"""
6     dxdt = (Vmax * S) / (Km + S) - kdeg * x
7     return dxdt
8
9 # Parametros nominais
10 params_nom = {'Vmax': 10, 'Km': 5, 'kdeg': 0.5, 'S': 8}
11
12 # Calcular sensibilidade por perturbacao
13 def calc_sensibilidade(param_name, delta=0.01):
14     """Calcula coeficiente de sensibilidade"""
15     # Valor nominal
```

Calculando Sensibilidade com Python II

```
16  params_base = params_nom.copy()
17  t = np.linspace(0, 50, 500)
18  x0 = 0
19  sol_base = odeint(modelo_sistema, x0, t,
20                  args=(params_base['Vmax'], params_base['Km',
21                        params_base['kdeg'], params_base['S',
22                        ]))
23
24  # Perturbar parametro
25  params_pert = params_base.copy()
26  params_pert[param_name] *= (1 + delta)
27  sol_pert = odeint(modelo_sistema, x0, t,
28                  args=(params_pert['Vmax'], params_pert['Km',
```

Calculando Sensibilidade com Python III

```
29             params_pert['kdeg'], params_pert['S',
30                 ]))
31
32     y_pert = sol_pert[-1, 0]
33
34     # Sensibilidade normalizada
35     S = ((y_pert - y_base) / y_base) / delta
36     return S
37
38 # Calcular para cada parametro
39 for param in ['Vmax', 'Km', 'kdeg']:
40     S = calc_sensibilidade(param)
41     print(f"S_{{x, {param}}}} = {S:.3f}")
```

Análise Global com SALib I

```
1 from SALib.sample import saltelli
2 from SALib.analyze import sobol
3 import numpy as np
4
5 # Definir problema
6 problem = {
7     'num_vars': 3,
8     'names': ['Vmax', 'Km', 'kdeg'],
9     'bounds': [[5, 15],      # Vmax: 50% variacao
10                [2.5, 7.5], # Km
11                [0.25, 0.75]] # kdeg
12 }
13
14 # Gerar amostras (metodo Saltelli para Sobol)
15 n_samples = 1024
```

Análise Global com SALib II

```
16 param_values = saltelli.sample(problem, n_samples)
17
18 # Avaliar modelo para cada conjunto de parametros
19 Y = np.zeros([param_values.shape[0]])
20
21 for i, params in enumerate(param_values):
22     Vmax, Km, kdeg = params
23     # Resolver ate estado estacionario
24     x_ss = (Vmax * 8) / (kdeg * (Km + 8))
25     Y[i] = x_ss
26
27 # Analise de Sobol
28 Si = sobol.analyze(problem, Y)
29
30 print("Indices de Sobol (efeitos principais):")
31 for name, s1 in zip(problem['names'], Si['S1']):
```

```
32     print(f"    S1_{name} = {s1:.3f}")
33
34 print("\nIndices de Sobol (efeitos totais):")
35 for name, st in zip(problem['names'], Si['ST']):
36     print(f"    ST_{name} = {st:.3f}")
```

Questão 1: Sistema Linear

Resolva analiticamente e numericamente (Python) o sistema:

$$\begin{aligned}\frac{dx}{dt} &= -x + 2y \\ \frac{dy}{dt} &= 2x - y\end{aligned}$$

com condições iniciais $(x_0, y_0) = (1, 0)$. Plote série temporal e retrato de fase.

Questão 2: Nulclinas

Para o sistema:

$$\begin{aligned}\frac{dx}{dt} &= x(2 - x - y) \\ \frac{dy}{dt} &= y(1 - x - 0.5y)\end{aligned}$$

1. Encontre e desenhe as nulclinas
2. Identifique todos os pontos de equilíbrio
3. Esboce o retrato de fase qualitativamente

Questão 3: Análise de Estabilidade

Analise a estabilidade de todos os equilíbrios do sistema:

$$\frac{dx}{dt} = y$$

$$\frac{dy}{dt} = -x - y + x^3$$

Calcule o Jacobiano, autovalores e classifique cada ponto fixo.

Questão 4: Lotka-Volterra Modificado

Considere o modelo com capacidade de carga:

$$\begin{aligned}\frac{dx}{dt} &= x(1 - x/K - \alpha y) \\ \frac{dy}{dt} &= y(-\delta + \beta x)\end{aligned}$$

com $K = 10$, $\alpha = 0.5$, $\beta = 0.3$, $\delta = 1$.

1. Encontre o equilíbrio não-trivial
2. Determine sua estabilidade
3. Simule e compare com Lotka-Volterra clássico

Questão 5: Diagrama de Bifurcação

Para $\dot{x} = \mu + x - x^3$, construa o diagrama de bifurcação completo:

1. Identifique o tipo de bifurcação
2. Determine valores críticos de μ
3. Plote ramos estáveis e instáveis
4. Gere o diagrama computacionalmente

Questão 6: Bifurcação de Hopf

Verifique numericamente a bifurcação de Hopf no sistema:

$$\begin{aligned}\frac{dx}{dt} &= \mu x - y - x(x^2 + y^2) \\ \frac{dy}{dt} &= x + \mu y - y(x^2 + y^2)\end{aligned}$$

Varie μ de -1 a 1 e observe surgimento do ciclo limite em $\mu = 0$.

Questão 7: Van der Pol

Simule o oscilador de Van der Pol para $\mu \in \{0.1, 1, 5\}$:

1. Compare amplitudes e formas de onda
2. Calcule período em cada caso
3. Plote retrato de fase para os três valores
4. Identifique regime de oscilações de relaxação

Questão 8: Modelo de Goodwin

Implemente o modelo de oscilador de Goodwin (3D):

$$\begin{aligned}\frac{dx}{dt} &= \frac{1}{1 + z^n} - k_1 x \\ \frac{dy}{dt} &= x - k_2 y \\ \frac{dz}{dt} &= y - k_3 z\end{aligned}$$

com $n = 4$, $k_1 = k_2 = k_3 = 1$. Mostre que existe ciclo limite.

Questão 9: Sensibilidade Local

Para o sistema de síntese e degradação:

$$\frac{dx}{dt} = k_s - k_d x^n$$

com $k_s = 1$, $k_d = 0.1$, $n = 2$:

1. Calcule analiticamente S_{x^*, k_s} , S_{x^*, k_d} , $S_{x^*, n}$
2. Verifique numericamente por perturbação
3. Qual parâmetro tem maior impacto?

Projeto Prático

Escolha um modelo de rede regulatória (ex: toggle switch, repressilator) e:

1. Implemente o modelo em Python
2. Realize análise de estabilidade completa
3. Identifique bifurcações variando um parâmetro chave
4. Conduza análise de sensibilidade global (Sobol)
5. Interprete biologicamente os resultados

Referências Principais I

-  Strogatz S.H. (2015). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*, 2nd ed. Westview Press.
-  Murray J.D. (2002). *Mathematical Biology I: An Introduction*, 3rd ed. Springer.
-  Keener J., Sneyd J. (2009). *Mathematical Physiology*, 2nd ed. Springer.
-  Edelstein-Keshet L. (2005). *Mathematical Models in Biology*. SIAM.
-  Alon U. (2019). *An Introduction to Systems Biology: Design Principles of Biological Circuits*, 2nd ed. CRC Press.

Livros Avançados

- Kuznetsov Y.A. (2004). *Elements of Applied Bifurcation Theory*, 3rd ed. Springer.
- Wiggins S. (2003). *Introduction to Applied Nonlinear Dynamical Systems and Chaos*, 2nd ed. Springer.
- Ermentrout G.B., Terman D.H. (2010). *Mathematical Foundations of Neuroscience*. Springer.

Artigos Fundamentais

- Tyson J.J. et al. (2003). The dynamics of cell cycle regulation. *BioEssays* 25:1095-1109.
- Novák B., Tyson J.J. (2008). Design principles of biochemical oscillators. *Nat Rev Mol Cell Biol* 9:981-991.
- Ferrell J.E. (2012). Bistability, bifurcations, and Waddington's epigenetic landscape. *Curr Biol* 22:R458-R466.

Software e Bibliotecas

- **Python:** `scipy.integrate` (`odeint`, `solve_ivp`), `sympy` (álgebra simbólica)
- **PyDSTool:** análise de sistemas dinâmicos, continuação, bifurcações
- **XPPAUT:** software clássico para EDOs e bifurcações
- **MATCONT:** continuação e bifurcações (MATLAB)
- **SALib:** análise de sensibilidade global (Python)

Tutoriais Online

- **SciPy Lecture Notes:** <https://scipy-lectures.org>
- **Dynamical Systems with Python:**
<https://github.com/cehberlin/dynamical-systems-python>
- **Strogatz's lectures:** YouTube (Cornell)

Obrigado!

Capítulo 4: The Mathematics of Biological Systems

Próximo Capítulo:
Estimação de Parâmetros e Ajuste de Modelos

Dúvidas? Perguntas?